# Mês 6 — Treino do Núcleo + Biblioteca v0 + Relatório Parcial (fev/2027)

Projeto PIBITI: Detecção de *Audio Deepfakes* usando WPT

Bolsista PIBITI

Fevereiro de 2027

# Sumário

**1 O que o plano exige** **2**

**2 GMM (Gaussian Mixture Model)** **3**
2.1 Abordagem de Duas Classes e Log-Likelihood Ratio . . . . . 3
2.2 Implementação Básica do GMM . . . . . 3
2.3 Calculando o Score (LLR) . . . . . 4

**3 XGBoost** **5**
3.1 Abordagem Discriminativa . . . . . 5
3.2 Predição e Probabilidades . . . . . 5
3.3 Feature Importance . . . . . 6

**4 Scores e thresholds** **7**

**5 EER (Equal Error Rate)** **8**
5.1 Cálculo do EER com sklearn . . . . . 8

**6 min-DCF** **9**

**7 Comparação GMM vs XGBoost** **10**

**8 Custo em CPU** **11**

**9 Biblioteca de extração v0** **12**
9.1 O arquivo `__init__.py` . . . . . 12
9.2 O arquivo `core.py` . . . . . 12

**10 Relatório parcial** **14**

**11 Curva ROC e Distribuição de Scores** **15**

**12 Brincadeiras: Experimentos Mão na Massa** **16**
12.1 Brincadeira 1: Extremos WPT . . . . . 16
12.2 Brincadeira 2: Outras Wavelets . . . . . 16
12.3 Brincadeira 3: Ajuste de GMM . . . . . 16
12.4 Brincadeira 4: XGBoost Learning Rate . . . . . 16
12.5 Brincadeira 5: Ptar Impacto no min-DCF . . . . . 16

**13 Checklist do Mês 6** **17**

**14 Desafio final** **18**

# Capítulo 1

# O que o plano exige

Neste mês, chegamos a um momento crucial do nosso projeto. O objetivo principal é validar o núcleo mínimo do sistema de detecção de *deepfakes*, entregar a versão inicial da biblioteca de extração (v0) e, muito importante, redigir o relatório parcial.

**Marco / Entrega**

- **Relatório parcial entregue**: Documento formalizando o progresso até aqui.
- **Núcleo validado**: Métricas de EER, min-DCF e custo computacional (CPU) devidamente calculadas e comparadas.
- **Biblioteca v0**: Um pacote básico e funcional para extrair features usando Wavelet Packet Transform (WPT).

**Atenção**

Para o cálculo do **min-DCF**, utilizaremos $C_{miss} = C_{fa} = 1$ e $P_{tar} = 0.05$. Devido ao desbalanceamento de classes típico deste cenário, daremos prioridade à métrica **EER** na avaliação geral de performance.

Neste mês, treinaremos dois modelos clássicos para classificar vetores de *features* extraídos via WPT: **GMM** (Gaussian Mixture Model) e **XGBoost**.

# Capítulo 2

# GMM (Gaussian Mixture Model)

O **Gaussian Mixture Model (GMM)** é um modelo probabilístico que assume que todos os pontos de dados são gerados a partir de uma mistura de um número finito de distribuições gaussianas com parâmetros desconhecidos.

## 2.1 Abordagem de Duas Classes e Log-Likelihood Ratio

Para verificação de voz e detecção de *spoofing*, a abordagem clássica é treinar dois GMMs independentes:

1. Um GMM treinado apenas com *features* de áudios autênticos (Bonafide).
2. Um GMM treinado apenas com *features* de áudios falsos (Spoof).

Para um novo segmento de áudio de teste, extraímos suas *features* e calculamos a *log-likelihood* (log-verossimilhança) em ambos os modelos. O score final é dado pela razão de log-verossimilhanças (Log-Likelihood Ratio - LLR):

$$Score = \log(P(X|\lambda_{bonafide})) - \log(P(X|\lambda_{spoof}))$$

## 2.2 Implementação Básica do GMM

Vamos usar o `sklearn.mixture.GaussianMixture` para treinar e avaliar.

```
import numpy as np
from sklearn.mixture import GaussianMixture

# Gerando dados ficticios (features extraidas)
np.random.seed(42)
X_bonafide = np.random.normal(loc=0.0, scale=1.0, size=(100, 20))
X_spoof = np.random.normal(loc=2.0, scale=1.5, size=(100, 20))

# Treinando os GMMs
gmm_bonafide = GaussianMixture(n_components=8, covariance_type='
    diag', random_state=42)
gmm_spoof = GaussianMixture(n_components=8, covariance_type='diag',
     random_state=42)

```

```
gmm_bonafide.fit(X_bonafide)
gmm_spoof.fit(X_spoof)
```

## 2.3 Calculando o Score (LLR)

```
def calcular_llr(gmm_bona, gmm_spf, X_teste):
    # .score_samples() retorna o log da densidade de probabilidade para cada amostra
    log_prob_bona = gmm_bona.score_samples(X_teste)
    log_prob_spf = gmm_spf.score_samples(X_teste)
    return log_prob_bona - log_prob_spf

X_teste = np.random.normal(loc=0.5, scale=1.2, size=(10, 20))
scores = calcular_llr(gmm_bonafide, gmm_spoof, X_teste)
print("Scores (LLR):", scores)
```

**Saída esperada**

```
Scores (LLR): [  8.32  12.14 -15.44 ...]
```

Scores muito positivos indicam forte chance de ser *bonafide*, enquanto scores negativos indicam *spoof*.

**Exercício**

Crie um script que instancie GMMs variando o número de componentes ($C \in \{2, 4, 8, 16\}$). Calcule o tempo de treinamento para cada configuração.

**Solução**

```
import time

for n_comp in [2, 4, 8, 16]:
    gmm = GaussianMixture(n_components=n_comp, covariance_type='diag')
    start = time.time()
    gmm.fit(X_bonafide)
    end = time.time()
    print(f"Componentes: {n_comp} | Tempo: {end - start:.4f}s")
```

# Capítulo 3

# XGBoost

**XGBoost** (eXtreme Gradient Boosting) é uma biblioteca otimizada de *gradient boosting* baseada em árvores de decisão.

## 3.1 Abordagem Discriminativa

Diferente do GMM (generativo), o XGBoost é discriminativo. Não treinamos um modelo por classe, mas sim um **único modelo** que tenta traçar uma fronteira de decisão entre *bonafide* e *spoof*.

```
import xgboost as xgb
from sklearn.model_selection import train_test_split

# Juntando dados para o XGBoost (0: spoof, 1: bonafide)
X = np.vstack((X_bonafide, X_spoof))
y = np.hstack((np.ones(len(X_bonafide)), np.zeros(len(X_spoof))))

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.2, random_state=42)

# Treinamento
modelo_xgb = xgb.XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=4,
    use_label_encoder=False,
    eval_metric='logloss'
)
modelo_xgb.fit(X_train, y_train)
```

## 3.2 Predição e Probabilidades

Ao invés de LLR, o XGBoost nos dá uma **probabilidade** de pertencer à classe positiva (*bonafide*, y=1).

```
# predict_proba retorna probabilidades para as classes [0, 1]
probs = modelo_xgb.predict_proba(X_test)
```

```
# Pegamos apenas a probabilidade da classe 1 (bonafide)
scores_xgb = probs[:, 1]
print("Probabilidades (Scores XGB):", scores_xgb[:5])
```

## 3.3 Feature Importance

O XGBoost permite inspecionar quais *features* foram mais determinantes:

```
importances = modelo_xgb.feature_importances_
print("Importancias:", importances[:5])
```

Isso é muito útil para entender quais bandas da *wavelet* estão carregando mais informação de *spoofing*.

# Capítulo 4

# Scores e thresholds

É fundamental entender a diferença entre a decisão de classe (0 ou 1) e o *score* (um número contínuo). Um *score* expressa a confiança do modelo.

Um **threshold** (limiar) é o valor de corte. Se *score* > *threshold*, classificamos como *bonafide*. Senão, como *spoof*.

```
threshold = 0.5
decisoes = (scores_xgb > threshold).astype(int)
print("Decisoes:", decisoes[:5])
```

Mudar o *threshold* afeta diretamente as taxas de Falso Aceite e Falsa Rejeição.

# Capítulo 5

# EER (Equal Error Rate)

**EER** é a métrica padrão-ouro em biometria e segurança. Ocorre quando:

$$FPR = FNR$$

Onde:

- FPR (False Positive Rate) = False Acceptance Rate (FAR): Spoof classificado como Bonafide.
- FNR (False Negative Rate) = False Rejection Rate (FRR): Bonafide classificado como Spoof.

## 5.1 Cálculo do EER com sklearn

```python
from sklearn.metrics import roc_curve
from scipy.optimize import brentq
from scipy.interpolate import interp1d

def calcular_eer(y_true, y_scores):
    fpr, tpr, thresholds = roc_curve(y_true, y_scores)
    fnr = 1 - tpr
    # O EER e o ponto onde FPR = FNR
    eer = brentq(lambda x : 1. - x - interp1d(fpr, tpr)(x), 0., 1.)
    thresh = interp1d(fpr, thresholds)(eer)
    return eer, thresh

eer_val, thresh_val = calcular_eer(y_test, scores_xgb)
print(f"EER: {eer_val*100:.2f}% no threshold {thresh_val:.4f}")
```

# Capítulo 6

# min-DCF

O custo mínimo da função de detecção (min-DCF, Minimum Detection Cost Function) considera que os erros têm custos diferentes.

Para a nossa tarefa: $C_{miss} = 1$, $C_{fa} = 1$ e probabilidade *a priori* do alvo (bonafide) $P_{tar} = 0.05$. O fato de $P_{tar}$ ser baixo reflete que a grande maioria dos ataques deve ser rejeitada, priorizando a segurança contra falsos alarmes de *spoof*.

```
def calcular_min_dcf(y_true, y_scores, p_tar=0.05, c_miss=1.0, c_fa=1.0):
    fpr, tpr, thresholds = roc_curve(y_true, y_scores)
    fnr = 1 - tpr

    # Custos nao normalizados
    dcf = c_miss * fnr * p_tar + c_fa * fpr * (1 - p_tar)

    # Custo de referencia (default system cost: ou rejeita todos ou aceita todos)
    dcf_ref = min(c_miss * p_tar, c_fa * (1 - p_tar))

    # DCF normalizado
    norm_dcf = dcf / dcf_ref

    return np.min(norm_dcf)

min_dcf_val = calcular_min_dcf(y_test, scores_xgb)
print(f"min-DCF: {min_dcf_val:.4f}")
```

# Capítulo 7

# Comparação GMM vs XGBoost

Aqui prepararemos o código base para gerar a tabela de resultados do relatório parcial.

```
# Vamos supor que treinamos e extraimos scores de teste para ambos
# LLR scores do GMM
y_scores_gmm = np.random.normal(0, 1, len(y_test)) # Simulacao
# Scores do XGBoost
y_scores_xgb = scores_xgb

eer_gmm, _ = calcular_eer(y_test, y_scores_gmm)
eer_xgb, _ = calcular_eer(y_test, y_scores_xgb)

dcf_gmm = calcular_min_dcf(y_test, y_scores_gmm)
dcf_xgb = calcular_min_dcf(y_test, y_scores_xgb)

print("==== TABELA DE RESULTADOS ====")
print(f"Modelo  | EER (%) | min-DCF")
print(f"GMM     | {eer_gmm*100:6.2f}  | {dcf_gmm:.4f}")
print(f"XGBoost | {eer_xgb*100:6.2f}  | {dcf_xgb:.4f}")
```

# Capítulo 8

# Custo em CPU

O custo computacional é vital num sistema WPT, já que a decomposição da árvore completa pode ser cara. Precisamos medir: 1. Tempo de Extração; 2. Tempo de Inferência.

```
import tracemalloc

def medir_inferencia_xgb(modelo, X):
    tracemalloc.start()
    t_inicio = time.time()

    preds = modelo.predict_proba(X)

    t_fim = time.time()
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    return t_fim - t_inicio, peak / 1024 / 1024 # MB

t_xgb, mem_xgb = medir_inferencia_xgb(modelo_xgb, X_test)
print(f"XGB - Inferencia de {len(X_test)} amostras:")
print(f"Tempo: {t_xgb:.4f}s | Memoria Pico: {mem_xgb:.4f} MB")
```

# Capítulo 9

# Biblioteca de extração v0

Nossa meta é empacotar o código desenvolvido nos meses anteriores. Estrutura sugerida para a entrega:

```
wpt_spoof_lib/
|-- __init__.py
|-- core.py
|-- utils.py
```

## 9.1 O arquivo `__init__.py`

```python
# wpt_spoof_lib/__init__.py
from .core import extrair_features

__version__ = "0.1.0"
__all__ = ["extrair_features"]
```

## 9.2 O arquivo `core.py`

```python
# wpt_spoof_lib/core.py
import pywt
import numpy as np

def extrair_features(audio, wavelet='db4', level=3):
    """Extrai features de energia WPT."""
    wp = pywt.WaveletPacket(data=audio, wavelet=wavelet, mode='
       symmetric', maxlevel=level)
    nodes = wp.get_level(level, 'freq')
    features = []
    for node in nodes:
        energy = np.sum(node.data ** 2)
        features.append(energy)
    return np.array(features)
```

**Exercício**

Crie um teste unitário básico (usando `assert`) que verifique se `extrair_features` com level 3 retorna um array de comprimento $2^3 = 8$.

**Solução**

```
def test_extracao():
    audio = np.random.randn(1000)
    feat = extrair_features(audio, level=3)
    assert len(feat) == 8, "Falha: deve retornar 8 features"
    print("Teste passou!")
test_extracao()
```

# Capítulo 10

# Relatório parcial

O relatório parcial a ser submetido ao CNPq/agência deve conter:

1. **Introdução**: Apresentar o problema dos *audio deepfakes* e a proposta de usar WPT.
2. **Referencial Teórico**: Wavelets, GMM, XGBoost e métricas (EER, min-DCF).
3. **Metodologia**: Como os áudios são lidos, a parametrização WPT, o setup de treino GMM/XGBoost.
4. **Resultados Preliminares**: As tabelas de métricas geradas e os gráficos ROC.
5. **Próximos Passos**: Integração com modelos profundos ou teste com banco de dados maior (ASVspoof).

# Capítulo 11

# Curva ROC e Distribuição de Scores

Visualizar a avaliação é fundamental para o relatório.

```
import matplotlib.pyplot as plt

# 1. Curva ROC
fpr, tpr, _ = roc_curve(y_test, scores_xgb)

plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, label=f'XGBoost (EER={eer_xgb*100:.2f}%)')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate (FAR)')
plt.ylabel('True Positive Rate (1 - FRR)')
plt.title('Curva ROC')
plt.legend()
plt.grid(True)
# plt.savefig('roc_curve.png')
# plt.show()
```

```
# 2. Distribuicao de Scores
scores_bona = scores_xgb[y_test == 1]
scores_spf = scores_xgb[y_test == 0]

plt.figure(figsize=(6,4))
plt.hist(scores_bona, bins=30, alpha=0.5, label='Bonafide')
plt.hist(scores_spf, bins=30, alpha=0.5, label='Spoof', color='red'
    )
plt.xlabel('Score (Probabilidade)')
plt.ylabel('Contagem')
plt.legend()
plt.title('Distribuicao de Scores')
# plt.show()
```

# Capítulo 12

# Brincadeiras: Experimentos Mão na Massa

Aqui estão 5 pequenos experimentos para você testar o pipeline inteiro.

## 12.1 Brincadeira 1: Extremos WPT

Altere o nível da wavelet de 3 para 5. O que acontece com a dimensão da *feature* $2^L$?

## 12.2 Brincadeira 2: Outras Wavelets

Teste `'sym8'` e `'coif2'`. O tempo de extração do WPT muda significativamente?

## 12.3 Brincadeira 3: Ajuste de GMM

Mude o tipo de covariância de `'diag'` para `'full'`.

```
gmm_full = GaussianMixture(n_components=4, covariance_type='full')
```

Note como o número de parâmetros a estimar cresce exponencialmente (possível overfitting se os dados forem poucos).

## 12.4 Brincadeira 4: XGBoost Learning Rate

```
modelo_lr = xgb.XGBClassifier(learning_rate=0.01, n_estimators=500)
```

Compare o desempenho com o anterior.

## 12.5 Brincadeira 5: Ptar Impacto no min-DCF

Modifique `p_tar` para 0.5. Como isso afeta o custo do modelo?

# Capítulo 13

# Checklist do Mês 6

Extração de features por pacote consolidado (`wpt_spoof_lib`).

Base em memória separada (Treino e Teste).

Treinamento GMM e cálculo de LLR.

Treinamento XGBoost e cálculo de probabilidades.

Funções implementadas para EER e min-DCF ($P_{tar} = 0.05$).

Gráficos salvos (ROC e Distribuição).

Redação do relatório concluída.

# Capítulo 14

# Desafio final

**Desafio**

O XGBoost nos mostra o *feature importance*. No contexto WPT, cada *feature* representa um "nó"da árvore, correspondendo a uma banda de frequência. **Sua missão**: Faça um script que mapeie os índices das *features* de maior importância no XGBoost para a faixa de frequência (em Hz) correspondente (assumindo uma taxa de amostragem de 16kHz). Discuta isso no relatório parcial!

```
# Esqueleto do desafio
fs = 16000
nivel = 3
largura_banda = (fs/2) / (2**nivel)
# Feature 0: 0 a largura_banda
# Feature 1: largura_banda a 2*largura_banda (Lembrando do
   ordenamento 'freq' do pywt)
```

*Bom trabalho com o relatório parcial! Você chegou longe e o núcleo do sistema está rodando. A base está lançada para os métodos mais avançados na próxima etapa.*